

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México, Escuela de Ingeniería y Ciencias.

Unidad de formación:

TC3009C.601

Inteligencia artificial avanzada para la ciencia de datos.

Actividad M2: Implementación de una técnica de aprendizaje máquina sin el uso de un framework.

Mario Alberto Pérez Barrera

A01799928

2 de Septiembre de 2026

Profesor: Jorge Adolfo Ramírez Uresti

Introducción

El presente trabajo implementa manualmente una red neuronal feedforward con una capa oculta, sin utilizar frameworks de aprendizaje automático (como scikit-learn, TensorFlow o PyTorch) para el algoritmo en sí. Se utiliza el algoritmo de retropropagación (backpropagation) junto con la función de pérdida de entropía cruzada binaria para resolver un problema de clasificación binaria.

Preparación del dataset

Se generó un dataset sintético controlado de 7,000 muestras con 2 características, mediante la función `make_classification` de scikit-learn (usada únicamente para crear los datos). El conjunto se dividió de forma estratificada en tres subconjuntos independientes:

Conjunto	Número de muestras	Porcentaje	Uso
Training	4200	60%	Ajuste de pesos y sesgos
Validation	1400	20%	Monitoreo durante el entrenamiento
Test	1400	20%	Evaluación del modelo

Arquitectura de la red neuronal

La red está compuesta por una capa de entrada de 2 nodos (correspondientes a las 2 características del dataset), una capa oculta de 4 neuronas y una capa de salida de 1 neurona. La función de activación utilizada en ambas capas es la sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Esta función es ideal para la capa de salida en clasificación binaria porque comprime cualquier valor real al rango (0, 1), interpretable como una probabilidad.

Inicialización de pesos y sesgos

Los pesos se inicializan aleatoriamente a partir de una distribución normal estándar (media 0, desviación estándar 1). Los sesgos se inicializan con valores unitarios. La inicialización aleatoria de los pesos es indispensable: si todos los pesos partieran del mismo valor, todas las neuronas de una misma capa aprenderían exactamente lo mismo y habría un problema de simetría por lo que la red no podría aprender representaciones distintas.

Forward propagation

El forward propagation transforma las entradas en una predicción para la capa de salida, en dos etapas: de la entrada a la capa oculta, y de la capa oculta a la salida.

$$\begin{aligned} z^{[1]} &= XW^{[1]} + b^{[1]} & a^{[1]} &= \sigma(z^{[1]}) \\ z^{[2]} &= a^{[1]}W^{[2]} + b^{[2]} & \hat{y} &= a^{[2]} = \sigma(z^{[2]}) \end{aligned}$$

Donde X es la matriz de entrada (tamaño $n_{\text{muestras}} \times 2$), $W^{[1]}$ y $W^{[2]}$ son las matrices de pesos de cada capa, $b^{[1]}$ y $b^{[2]}$ son los vectores de sesgo, y la salida de la red es la probabilidad predicha de que la muestra pertenezca a la clase 1. Cada z se calcula como una combinación lineal (producto matricial de entradas por pesos, más el sesgo) y luego se le aplica la función de activación sigmoide para introducir no linealidad.

Función de pérdida

Para cuantificar la discrepancia entre las predicciones y las etiquetas reales se utilizó la entropía cruzada binaria:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Backpropagation y actualización de pesos

El entrenamiento se basa en descenso de gradiente: en cada época se calcula qué tanto contribuyó cada peso y cada sesgo al error final, y se ajustan en la dirección que reduce ese error. El gradiente se propaga desde la salida hacia la entrada aplicando la regla de la cadena en forma matricial.

Error en la capa de salida

Gracias a que se combinó la sigmoide con la entropía cruzada, la derivada de la pérdida respecto a z se simplifica a una resta directa entre la predicción y la etiqueta real:

$$\delta^{(2)} = \hat{y} - y$$

Error con backpropagation hacia la capa oculta

El error de la capa de salida se reparte hacia atrás multiplicándose por la transpuesta de los pesos de salida, y se multiplica elemento a elemento por la derivada de la sigmoide en la capa oculta:

$$\delta^{(1)} = (\delta^{(2)} (W^{(2)})^T) \odot (a^{(1)} (1 - a^{(1)}))$$

Gradientes y actualización

Con los errores (δ) de cada capa, los gradientes respecto a los pesos y sesgos se obtienen como productos matriciales: $\partial L / \partial W = (\text{activación de la capa anterior})^T \cdot \delta$, y $\partial L / \partial b = \text{suma de } \delta \text{ sobre las muestras del batch}$ (ambos divididos entre el número de muestras N). Finalmente, los pesos y sesgos se actualizan en la dirección opuesta al gradiente.

$$W = W - \alpha \frac{\partial L}{\partial W} \qquad b = b - \alpha \frac{\partial L}{\partial b}$$

Resultados del entrenamiento

Se ejecutaron 100 épocas con una tasa de aprendizaje $\alpha = 0.02$. La tabla siguiente muestra la evolución de la pérdida y la exactitud cada 10 épocas:

```
Epoch 10/100 - Train Loss: 0.9095, Train Acc: 0.499, Val Loss: 0.8953, Val Acc: 0.499
Epoch 20/100 - Train Loss: 0.8095, Train Acc: 0.499, Val Loss: 0.7972, Val Acc: 0.499
Epoch 30/100 - Train Loss: 0.7279, Train Acc: 0.499, Val Loss: 0.7174, Val Acc: 0.499
Epoch 40/100 - Train Loss: 0.6629, Train Acc: 0.499, Val Loss: 0.6540, Val Acc: 0.499
Epoch 50/100 - Train Loss: 0.6122, Train Acc: 0.499, Val Loss: 0.6046, Val Acc: 0.499
Epoch 60/100 - Train Loss: 0.5732, Train Acc: 0.499, Val Loss: 0.5666, Val Acc: 0.499
Epoch 70/100 - Train Loss: 0.5432, Train Acc: 0.520, Val Loss: 0.5375, Val Acc: 0.535
Epoch 80/100 - Train Loss: 0.5203, Train Acc: 0.713, Val Loss: 0.5151, Val Acc: 0.751
Epoch 90/100 - Train Loss: 0.5025, Train Acc: 0.907, Val Loss: 0.4977, Val Acc: 0.922
Epoch 100/100 - Train Loss: 0.4886, Train Acc: 0.971, Val Loss: 0.4841, Val Acc: 0.967
```

Accuracy de la prueba:

Accuracy en test: 0.9700

Se observa un comportamiento característico: durante las primeras 65 épocas la red se mantiene estancada en 49.9% de exactitud, y a partir de la época 70 ocurre una transición rápida hasta estabilizarse cerca del 97%.

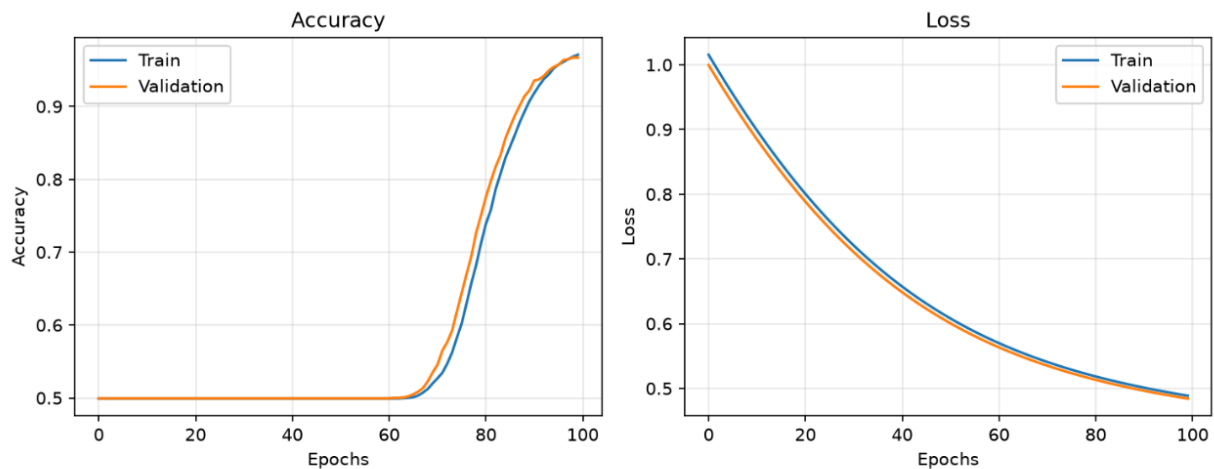


Figura 1. Curvas de accuracy y pérdida en entrenamiento y validación

La cercanía entre las curvas de entrenamiento y validación, tanto en pérdida como en exactitud, indica que el modelo generaliza adecuadamente y no presenta overfitting.

Resultados del test

Para el desempeño final se evaluó sobre el conjunto de pruebas de 1400 muestras.

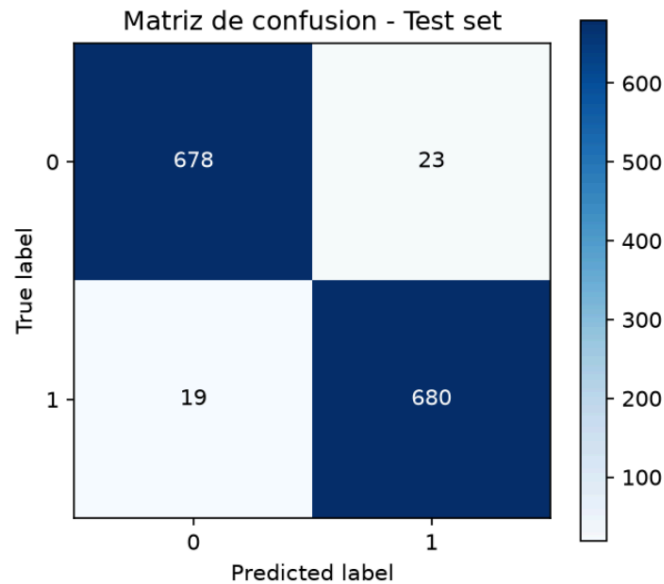


Figura 2. Matriz de confusión

De un total de 1,400 instancias de prueba, se cometieron 42 errores de clasificación: 23 falsos positivos y 19 falsos negativos. Las 1,358 clases restantes fueron clasificadas correctamente.

Classification report:				
	precision	recall	f1-score	support
0	0.973	0.967	0.970	701
1	0.967	0.973	0.970	699
accuracy			0.970	1400
macro avg	0.970	0.970	0.970	1400
weighted avg	0.970	0.970	0.970	1400

Figura 3. Reporte de clasificación en el conjunto de prueba.

El modelo alcanzó una accuracy del 97%, con precisión y recall equilibrados entre ambas clases (F1-score de 0.970 en ambos casos). Podemos confirmar que el modelo no favorece a ninguna clase por encima de la otra.

Análisis y conclusión

El desempeño en el conjunto de prueba (97.0%) es consistente con el desempeño en validación (96.7%) y muy cercano al de entrenamiento (97.1%). La ausencia de una brecha significativa entre estos tres valores indica un modelo bien ajustado (fit): ni con sesgo alto, no hay underfitting, la exactitud es alta en los tres conjuntos, ni con varianza alta (no hay overfitting, el desempeño no se degrada en datos no vistos). En conclusión, la implementación manual de la red neuronal feedforward logró un desempeño alto y estable (97% de exactitud) en un problema de clasificación binaria sintética, validando que los cálculos de forward propagation, función de pérdida y backpropagation fueron implementados correctamente.

Referencias

Geeks for Geeks. (2025). Implementation of neural network from scratch using NumPy. <https://www.geeksforgeeks.org/numpy/implementation-of-neural-network-from-scratch-using-numpy/> Nayar, S. (2021). Backpropagation Algorithm | Neural Networks. [Archivo de video]. YouTube. https://www.youtube.com/watch?v=sIX_9n-1UbM